- We used a transaction to help prevent deletion of the last user.

Playtime

Here's some stuff to try on your own:

- Modify the user update function to require and validate the current password before allowing a user's password to be changed.

- The system test in `test/system/users_test.rb` was generated by the scaffolding generator we used at the start of the chapter. Those tests don't pass. See if you can get them to pass without breaking the other system tests. You'll recall we created the module `AuthenticationHelpers` and included it in all of the system tests by default, so you might need to change the code to *not* do that so that you can properly test the login functionality.

  When the system is freshly installed on a new machine, no administrators are defined in the database, and hence no administrator can log on. But if no administrator can log on, then no one can create an administrative user.

  Change the code so that if no administrator is defined in the database, any username works to log on (allowing you to quickly create a real administrator).

- Experiment with `rails console`. Try creating products, orders, and line items. Watch for the return value when you save a model object—when validation fails, you'll see `false` returned. Find out why by examining the errors:

```
>> prd = Product.new
=> #<Product id: nil, title: nil, description: nil, image_url:
nil, created_at: nil, updated_at: nil, price:
#<BigDecimal:246aa1c,'0.0',4(8)>>
>> prd.save
=> false
>> prd.errors.full_messages
=> ["Image url must be a URL for a GIF, JPG, or PNG image",
    "Image url can't be blank", "Price should be at least 0.01",
    "Title can't be blank", "Description can't be blank"]
```